

Rapport Technique d'Implémentation

Simulateur de Diagnostic de Pannes dans les Réseaux Optiques par
Combinatorial Group Testing (CGT)

Oussama HAKIK

25 mars 2026

Résumé

Ce rapport détaille l'implémentation d'un simulateur de diagnostic de pannes pour réseaux optiques, s'appuyant sur la théorie du Combinatorial Group Testing (CGT). L'objectif est d'isoler une défaillance unique (arête coupée) avec un minimum de sondes, soit une complexité de $O(\log N)$. Nous exposons d'abord les topologies Linéaire, Grille et Complet. Ensuite, nous vulgarisons et détaillons la topologie "Arbre", qui utilise la Décomposition Lourd-Léger. Enfin, nous expliquons comment le comportement physique de la lumière (le laser qui se bloque) vient fausser la théorie mathématique, et comment les algorithmes corrigent ce problème pour rendre le diagnostic infaillible.

Table des matières

1	Introduction et Contexte	2
1.1	La Matrice Binaire (Le Plan de Test)	2
2	Les Topologies de Base	2
2.1	Topologie Linéaire	2
2.2	Topologie Grille ($n \times n$)	2
2.3	Topologie Complète (Hub & Cycles)	4
3	Le Cœur de l'article : La Topologie Arbre	4
3.1	Étape 1 : Arêtes Lourdes (Autoroutes) et Légères (Sorties)	5
3.2	Étape 2 : Tracer les Autoroutes	5
4	La Théorie Mathématique face au "Laser Bloqué"	6
4.1	Ma Solution : Le Décodage de Haut en Bas (Top-Down)	7
4.2	L'Élimination (Intersection Spatiale)	7
5	Conclusion	7

1 Introduction et Contexte

Trouver une panne dans un réseau de fibre optique ressemble souvent à chercher une aiguille dans une botte de foin. La méthode classique (naïve) consiste à tester les liens un par un, ce qui prend énormément de temps ($O(N)$ tests).

Pour aller beaucoup plus vite, nous utilisons le **Combinatorial Group Testing (CGT)**. Le principe est simple : au lieu de tester un lien à la fois, un seul tir de laser va traverser tout un groupe de liens en même temps. En croisant les résultats de plusieurs groupes, on devine mathématiquement où est la panne. Nous utilisons ici une méthode **statique** : tous les tirs de lasers sont planifiés à l'avance et lancés en même temps.

1.1 La Matrice Binaire (Le Plan de Test)

L'algorithme génère un tableau (une matrice) rempli de 0 et de 1. Chaque ligne de ce tableau représente un tir de laser (une sonde). Si la colonne d'un lien contient un "1", le laser doit passer par ce lien. Si le laser ne revient pas, on note 1 (Échec). S'il revient, on note 0 (Succès). En mettant ces 1 et ces 0 bout à bout, on obtient un mot binaire appelé **Syndrome**. En convertissant ce mot en nombre décimal, on trouve directement le numéro de la panne.

2 Les Topologies de Base

2.1 Topologie Linéaire

Les nœuds sont alignés à la queue leu leu. L'algorithme utilise une "Fenêtre Glissante" pour tester des gros segments de la ligne. L'intersection des fenêtres en échec donne la panne.

2.2 Topologie Grille ($n \times n$)

Pour trouver une panne dans un quadrillage, on procède comme à la bataille navale :

1. On teste des lignes ou des colonnes entières d'un seul coup.
2. Une fois la ligne coupée identifiée, on utilise un test en "échelle" (qui teste le k -ième élément de chaque ligne) pour trouver l'endroit exact.

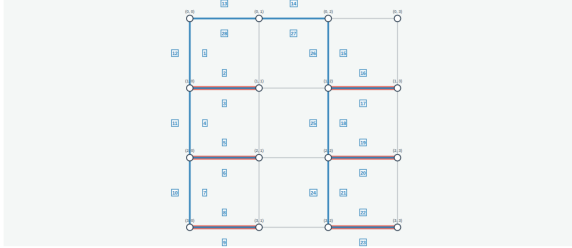


FIGURE 1 – Arêtes ciblées sur la grille.



FIGURE 2 – Matrice binaire générée.

FIGURE 3 – Ici à droite, on est dans la phase où on teste la k -ième arête de toutes les lignes. En appliquant le **LTP**, le premier test T1 (ligne 1 du tableau) va envoyer le laser cibler la 1ère arête de toutes les lignes (0) et la 3ème arête (2). À gauche, on voit en gras les arêtes cibles.

L'intersection des résultats permet de cibler la panne :

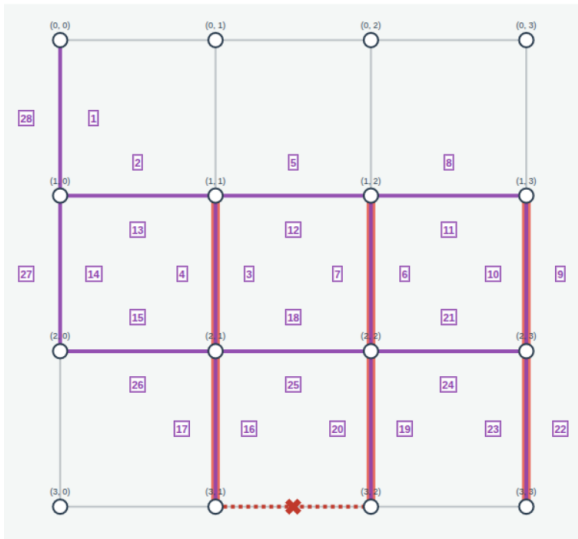


FIGURE 4 – Schéma physique de la panne.



FIGURE 5 – Rapport de déduction mathématique.

FIGURE 6 – En croisant le test des Lignes Globales et le test des Échelles, le contrôleur déduit l'emplacement exact de la panne.

2.3 Topologie Complète (Hub & Cycles)

Ici, tout le monde est connecté à tout le monde. L'algorithme procède en deux temps :

- **Phase 1 (Étoile)** : On teste les liens directs vers le nœud central.
- **Phase 2 (Cycles)** : On utilise le centre comme point de rebond pour tester les liens distants, en construisant un circuit optique (Circuit Eulérien) qui fait une boucle.

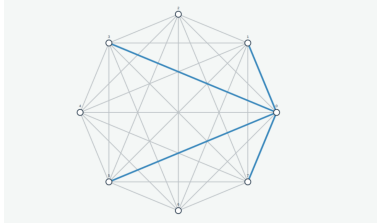


FIGURE 7 – Topologie Complète.

Status de l'Algorithme

ÉTAPE : STAR-LTP-1 - LTP Hub

DESCRIPTION :
Test combinatoire sur les nœuds [1, 3, 5, 7].

STATUT : ✔ SUCCÈS

MATRICE : HUB (Nœuds)

Sonde	1	2	3	4	5	6	7
T1 (bit 0)	1	0	1	0	1	0	1
T2 (bit 1)	0	1	1	0	0	1	1
T3 (bit 2)	0	0	0	1	1	1	1

FIGURE 8 – Matrice Hub.



FIGURE 9 – Chemin de la sonde T1.

FIGURE 10 – Processus sur un graphe complet : structure physique (gauche), matrice binaire (centre), et génération du circuit optique (droite).

Si la panne se trouve sur une arête distante, la matrice des cycles nous donne le code d'erreur :

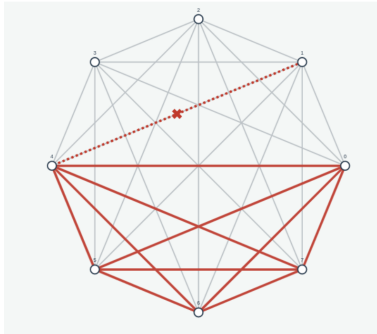


FIGURE 11 – Panne sur l'arête 1-4.

MATRICE : CYCLES (ARÊTES)

Sonde	(1,3)	(1,4)	(1,5)	(1,7)	(2,3)	(2,4)	(2,5)	(2,7)	(3,4)	(3,5)	(3,7)	(4,5)	(4,7)
T1 (bit 0)	1	1	1	1	1	1	1	1	1	1	1	1	1
T2 (bit 1)	1	1	1	1	1	1	1	1	1	1	1	1	1
T3 (bit 2)	1	1	1	1	1	1	1	1	1	1	1	1	1
T4 (bit 3)	1	1	1	1	1	1	1	1	1	1	1	1	1
T5 (bit 4)	0	0	0	0	0	0	0	0	0	0	0	1	1

FIGURE 12 – Matrice des Cycles.

Rapport du Contrôleur (Décodage Top-Down)

Le contrôleur a collecté l'ensemble des statuts 0 et 1 pour effectuer ses calculs :

Analyse Matrice : Cycles (Arêtes)

- Vecteur binaire lu (T_n ... T₁) : 00011
- Déduction Théorique : L'élément fautif correspond à l'index 3 => **14**

FIGURE 13 – Analyse des résultats.

FIGURE 14 – Le code lu de bas en haut donne 00011 (soit 3 en décimal). La panne est le 3ème élément de la liste des arêtes.

3 Le Cœur de l'article : La Topologie Arbre

Diagnostiquer un réseau en forme d'arbre est très compliqué car il y a des branches dans tous les sens. La solution consiste à utiliser une méthode appelée **Heavy-Light Decomposition (HLD)**. Pour simplifier, le but de cette méthode est de "découper" l'arbre pour le transformer en un ensemble de lignes droites (que nous appellerons des "**Autoroutes**").

3.1 Étape 1 : Arêtes Lourdes (Autoroutes) et Légères (Sorties)

Imaginez que l'on cherche à tracer les plus grandes routes possibles en partant du centre. L'algorithme va regarder chaque nœud (chaque croisement) et compter combien il y a de nœuds en dessous de lui. Le lien qui mène vers le chemin le plus long/peuplé devient un lien **Lourd** (un trait plein). Les petits liens qui partent sur les côtés deviennent des liens **Légers** (des pointillés, comme des bretelles de sortie).

Algorithm 1 Calcul des Poids et Classification des Arêtes

Rôle : Déterminer le chemin principal de chaque branche.

Explication : L'algorithme descend dans l'arbre. Pour chaque croisement, il pèse les différentes branches. La branche la plus lourde devient la continuité de l'autoroute, les autres deviennent des sorties.

```
1: Fonction CalculerPoids(nœud, parent)
2: Poids  $\leftarrow$  1
3: MaxPoids  $\leftarrow$  0, EnfantLourd  $\leftarrow$  Rien
4: for chaque voisin connecté au nœud do
5:   if voisin  $\neq$  parent then
6:     PoidsEnfant  $\leftarrow$  CalculerPoids(voisin, nœud)
7:     Poids  $\leftarrow$  Poids + PoidsEnfant
8:     if PoidsEnfant > MaxPoids then
9:       MaxPoids  $\leftarrow$  PoidsEnfant
10:      EnfantLourd  $\leftarrow$  voisin
11:   end if
12: end if
13: end for
14: Marquer le lien vers EnfantLourd comme Lourd (Trait plein)
15: Marquer les autres liens comme Légers (Pointillés)
16: return Poids
```

3.2 Étape 2 : Tracer les Autoroutes

Maintenant qu'on a classé les liens, on trace nos lignes droites. Une "Autoroute" se crée en suivant les liens lourds. Dès qu'on prend un lien léger (une sortie), on démarre une nouvelle Autoroute.

Le Piège à éviter : Si on commence la nouvelle autoroute *après* la sortie, le lien de la bretelle (en pointillés) n'appartiendra à aucune autoroute. Le laser l'oubliera! **La Solution :** L'algorithme a été conçu pour que **chaque nouvelle autoroute commence toujours par sa propre bretelle d'accès**. Même si la sortie mène vers un cul-de-sac, elle sera considérée comme une mini-autoroute à elle toute seule.

Algorithm 2 Création des Autoroutes (Chemins Préférés)

Rôle : Découper l'arbre tordu en lignes droites parfaites pour la matrice.

Explication : On commence avec la bretelle d'accès, puis on avance tant qu'il y a un trait plein. Dès qu'on voit des traits en pointillés sur les côtés, on met en pause et on lance la création d'une nouvelle autoroute pour ces sorties.

```
1: Fonction TracerAutoroute(nœud_depart, bretelle_legere)
2: Autoroute  $\leftarrow$  [bretelle_legere, nœud_depart]
3: nœud_actuel  $\leftarrow$  nœud_depart
4: while Il y a un lien Lourd devant le nœud_actuel do
5:   nœud_suivant  $\leftarrow$  nœud au bout du lien lourd
6:   Ajouter nœud_suivant à l'Autoroute
7:   nœud_actuel  $\leftarrow$  nœud_suivant
8: end while
9: Sauvegarder cette Autoroute dans notre plan de test.
10: for chaque nœud sur cette nouvelle Autoroute do
11:   for chaque lien Léger (sortie) sur le côté do
12:     TracerAutoroute(nœud_de_sortie, cable_leger)
13:   end for
14: end for
```

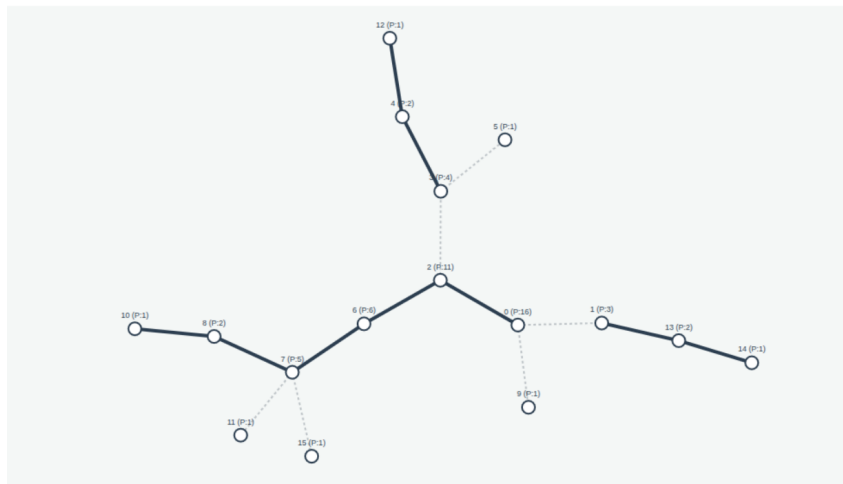


FIGURE 15 – Résultat visuel : L'arbre est découpé en Autoroutes. L'autoroute principale part de 0 jusqu'à 10 (traits pleins). La sortie en pointillés (0 vers 9) est un cul-de-sac, elle devient l'Autoroute N°1.

4 La Théorie Mathématique face au "Laser Bloqué"

L'article de recherche dont nous nous inspirons suppose que la lumière est magique. Dans sa théorie, on peut tester l'autoroute N°4 sans aucun problème, même si elle est tout au bout de l'arbre.

Le Vrai Problème (L'Effet Cascade) :

Dans la vraie vie, le laser part du centre (la Racine). Pour aller tester l'autoroute N°4, il est obligé de rouler sur l'autoroute N°0. Que se passe-t-il si l'autoroute N°0 est coupée ? Le laser est bloqué dès le début ! Il n'atteindra jamais la N°4. La matrice mathématique va voir que le laser

n'est pas revenu, et va accuser à tort l'autoroute N°4, alors que le vrai coupable est l'autoroute N°0. La théorie mathématique pure se fait tromper par la physique.

4.1 Ma Solution : Le Décodage de Haut en Bas (Top-Down)

Pour empêcher le simulateur de se tromper, nous avons classé les autoroutes par "Strates" (Niveaux d'éloignement) :

- **Strate 0** : Les autoroutes directes (sans prendre de sortie).
- **Strate 1** : Les autoroutes où il a fallu prendre 1 sortie.

À la fin du test, au lieu de tout lire d'un coup, **l'algorithme lit les résultats de Haut en Bas**. Il lit la Strate 0. S'il trouve une panne, il s'arrête net. Il jette les résultats des strates inférieures à la poubelle, car il a compris que le laser y a été bloqué en chemin.

4.2 L'Élimination (Intersection Spatiale)

Pour être sûr à 100% de l'endroit exact de la panne :

1. On prend tous les liens où le laser n'est pas passé (Les suspects).
2. On retire les liens où d'autres lasers ont réussi à passer (Les innocents).

La Règle du Goulot d'Étranglement : S'il reste encore plusieurs suspects à la suite, la coupure est obligatoirement le lien **le plus proche du centre**. C'est lui le vrai coupable qui a bloqué la route aux autres !

5 Conclusion

Ce simulateur prouve que l'on peut trouver une panne très rapidement grâce aux mathématiques (Matrice Binaire [LTP]). Cependant, il démontre aussi que la théorie pure a ses limites. En introduisant des algorithmes pratiques comme le décodage de Haut en Bas et l'élimination spatiale, nous avons corrigé les failles de la théorie pour rendre le diagnostic physique totalement infaillible (après je ne dit pas que j'ai raison de ce que j'ai fait jusqu'à maintenant x) avant la discussion avec vous).